Learning to Play Pursuit-Evasion with Dynamic and Sensor Constraints

Burak M. Gonultas[1] and Volkan Isler[1]

***Abstract*— We present a multi-agent reinforcement learning approach to solve a pursuit-evasion game between two players with car-like dynamics and sensing limitations. We develop a curriculum for an existing multi-agent deterministic policy gradient algorithm to simultaneously obtain strategies for both players, and deploy the learned strategies on real robots moving as fast as 2 m/s in indoor environments. Through experiments we show that the learned strategies improve over existing baselines by up to 30% in terms of capture rate for the pursuer. The learned evader model has up to 5% better escape rate over the baselines even against our competitive pursuer model. We also present experiment results which show how the pursuit-evasion game and its results evolve as the player dynamics and sensor constraints are varied. Finally, we deploy learned policies on physical robots for a game between the F1TENTH and JetRacer platforms and show that the learned strategies can be executed on real-robots. Our code and supplementary material including videos from experiments are available at `https://gonultasbu.github.io/pursuit-evasion/`.**

Fig. 1: **Real-world experiments:** Execution of the learned pursuit-evasion policies on two robotic cars with varying wheelbase, velocity and steering angle constraints.

I. Introduction

Pursuit-evasion games are optimization problems in which one or more pursuers try to "capture" an evader. The notion of capture is usually formulated as being co-located or getting close. However there are other formulations such as visibility-based pursuit evasion [1], [2] where capture is achieved when the evader is located. In robotics, pursuit-evasion games are studied to model motion planning problems in dynamic scenarios and to investigate the coupling between player dynamics, observation models and environment complexity.

As an example, in the classical homicidal chauffeur game [3], the pursuer is a car-like vehicle whose goal is to hit a pedestrian who is not subject to any turning constraints. In this game, the pursuer has three state variables (position and orientation) and the evader has two (only position). One might view the solution of this game as a subspace (manifold) of this five dimensional joint state space. The tangent space of this solution manifold corresponds to the optimal control actions of the players. In some cases, the differential equations for the evolution of the game when the players play optimally can be derived in closed form. The strategies for a given instance is usually obtained by numerical integration. Alternatively, the state space can be discretized and the solution can be obtained using dynamic programming techniques.

However, such methods become quickly intractable as the dimensionality of the game increases. For example, in his book, Isaacs presents only a partial solution for the "game of two cars" where both players have car-like dynamics. The dimensionality issue becomes more severe when sensing is involved: in the standard formulations, it is assumed that the players can observe each others' state fully at each time step. In practice, this may not be always true due to sensor and observability limitations. The resulting "partial observability games" have even higher (sometimes infinite) dimensionality as the players now need to reason about all feasible states that are consistent with their observation histories.

Reinforcement Learning (RL) [4] methods provide an alternative approach to construct the solution manifold by learning it through game play. Modern RL methods often represent the solution as a neural network which takes the observations as input and returns a distribution over the actions. In this paper, we investigate whether recently developed multi-agent reinforcement learning (MARL) techniques can solve a challenging game where the pursuer is a car with five state variables including velocities. We investigate two versions of the evader. It is either a car with the same kinematic model or a point mass. Hence, the joint-state dimensionality can be as high as 10. The players are subject to field-of-view sensing restrictions for both the viewing angle and viewing distance (Section III). We cast the game as a competitive Partially Observable Stochastic Game (POSG) and use the MARL algorithm *multi-agent deep deterministic policy gradient (MADDPG)* [5] to simultaneously train pursuer and evader policies. Our problem formulation and proposed approach do not require any pretrained or designed expert evader policy and generalizes over varying agent constraints.

Our contributions are as follows:

[1] the authors are with the Department of Computer Science and Engineering, University of Minnesota, Minneapolis, MN, 55455, USA {gonul004, isler}@umn.edu

This work was supported by the MN LCCMR program.

- We develop a curriculum strategy for MADDPG to solve a pursuit-evasion game with complex dynamics and sensing constraints. We show that our method is able to train competitive pursuer and evader policies without the need for expert policies.
- We provide insights on how the pursuit-evasion game and its results evolve as the player dynamic and sensor constraints are varied.
- In simulation experiments we individually compare pursuer and evader policies against existing baselines.
- We deploy learned policies on physical robots for a game between the F1TENTH [6] and JetRacer [7] platforms moving at high-speed and showcase a successful policy transfer to real world.

Overall, our results show the potential of multi-agent reinforcement learning in navigating complex pursuit-evasion scenarios within dynamic environments and demonstrate that simulation-trained policies are directly transferable to real autonomous agents.

II. Related Work

In this section, we review relevant pursuit-evasion and MARL literature.

A. Pursuit-Evasion

The mathematical origins of the pursuit-evasion problem can be traced back to 1930's as the Lion and Man problem. In the original formulation, a lion and a human are in a circular area where the lion tries to catch the human as quickly as possible when the human tries to avoid the lion for as long as possible. Since then, various variants of this game have been studied in the robotics literature [8]. In his seminal work [3] Isaacs presented a differential game based formulation to the pursuit-evasion problem which has served as the basis of game-theoretic approaches.

In control-theoretic formulations, both the pursuer and evader in a pursuit-evasion problem have constraints on maximum velocities. One prominent example is the Homicidal chauffeur problem formulated by Isaacs [3], which is a special case of the pursuit-evasion problem where the pursuer has nonholonomic constraints. Since then, numerous works have studied pursuit-evasion games with dynamic constraints. In [9], the authors study the "Suicidal Pedestrian problem" inspired by Isaacs. Ruiz and Murrieta-Cid [10] analyze a scenario with a differential drive, faster evader against an omnidirectional but slower pursuer. In [11] Scott and Leonard derive and analyze optimal control strategies for speed, turning rate, lateral acceleration and movement direction-constrained evaders. In [12] the authors propose a single-agent reinforcement learning approach to the multi-pursuer with unicycle constraints scenario against a single, omnidirectional evader. Similarly, in [13] Yang et al. study a multi-pursuer, multi-evader version of pursuit-evasion problem with unicycle constraints and propose a MARL state processing method to improve the scalability.

If the players can not observe each other at all times, for example, if the game is played in a complex environment under sensing limitations, pursuit-evasion game has been defined as a combination of two subcomponents: 1) locating the evader (search problem) and 2) capturing the evader. From the pursuer perspective, the search problem appears once sensor constraints are introduced to the pursuit-evasion problem. From the evader perspective, optimal strategies depend on the environment as well as the sensor constraints of the agents in the pursuit-evasion game. The problem of planning paths to reach a state with unobstructed view of the evader was first presented in [14]. In [1], it was shown that in a simply connected polygon of n vertices, $\Theta(\log n)$ pursuers are necessary and sufficient to detect an unpredictable evader. Numerous studies since then have formulated different variations of the pursuit-evasion problem, [15]–[17] with [15] studying the case with a pursuer with limited field of view and [16] with unreliable sensor data. In [2] Isler et al. propose a randomized algorithm to locate the evader with a single pursuer and extend this strategy to two pursuers with line-of-sight visibility to capture the evader in simply connected polygons. In later work, [18] Noori and Isler have shown that a single pursuer can locate and capture the evader in monotone polygons. In [19] Engin et al. have proposed a single-agent reinforcement learning method with a compact belief state representation for pursuit-evasion games with visibility constraints in simply-connected polygons. In [20] Bajcsy et al. have studied the pursuit-evasion problem for quadrupedal robots under real-world physical and visibility constraints and proposed a teacher-student network pair to deal with the issues caused by partial observability in the problem definition.

In this work, we investigate a game between players with complex dynamics *and* with sensing limitations.

B. Dynamic Games & MARL

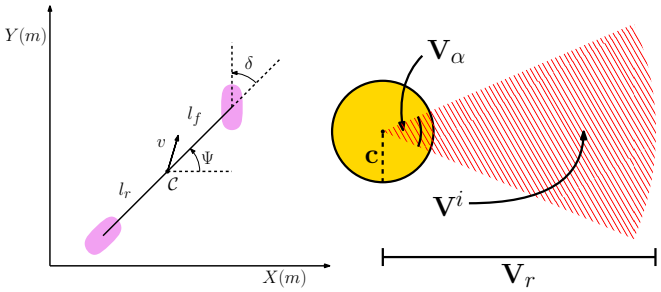
Isaacs' [3] and other game-theoretic formulations [21], [22] have long influenced robotics, controls and reinforcement learning research [23]–[26]. Significant progress has been made using MARL algorithms [5], [27]–[29] in settings where large-scale simulations are feasible such as Starcraft, Dota 2, Go and Chess [30]–[33]. Although there are recent works demonstrating successful reinforcement learning methods' deployment on physical systems [20], [34], joint learning in game settings and deployment on physical systems still remains a challenge for the robotics community.

III. Problem Formulation

In this section, we introduce the relevant terminology and models used in our problem definition.

A. Agent and Environment Models

The pursuer kinematic model, which is represented by the state-space model [35], is given in Eq (1). It consists of the



(a) (b)

Fig. 2: **Car kinematic bicycle model and sensor representations:** For simulated experiments, the pursuer uses the kinematic bicycle model presented in (a). For physical experiments, both agents use the kinematic bicycle model with different parameters. All agents use the same sensor model presented in (b).

following states $x_1 = s_x, x_2 = s_y, x_3 = \delta, x_4 = v, x_5 = \Psi$.

$$\begin{aligned} \dot{x_1} &= x_4 cos(x_5) \\ \dot{x_2} &= x_4 sin(x_5) \\ \dot{x_3} &= f_{steer}(x_3, u_1) \\ \dot{x_4} &= f_{acc}(x_4, u_2) \\ \dot{x_5} &= \frac{x_4}{l_f + l_r} tan(x_3) \end{aligned} \tag{1}$$

where s_x and s_y are the coordinates in meters in the global frame. v is the longitudinal velocity in m/s, and Ψ is the global yaw angle at the vehicle center in radians. Inputs are defined as u_1 representing the steering velocity $\dot{\delta}$ in rad/s and u_2 representing the longitudinal acceleration $\dot{v}$ in m/s^2. System steering and acceleration input constraints are represented as f_{steer} and f_{acc}, respectively. $l_f + l_r$ is the car model parameter representing the vehicle wheelbase with l_f representing the distance of the front axle from the center of gravity and l_r representing the same for the rear axle in meters. See Fig. 2a.

The evader is based on the point-mass state-space model [35] defined in Eq. (2), consisting of the following states $x_1 = s_x, x_2 = s_y, x_3 = \dot{s_x}, x_4 = \dot{s_y}$ and uses the same global frame of reference notation with the inputs u_1 and u_1 representing the accelerations along the global x and y-axes $\ddot{s_x}$ and $\ddot{s_y}$ in m/s^2.

$$\begin{aligned} \dot{x_1} &= x_3 \\ \dot{x_2} &= x_4 \\ \dot{x_3} &= u_1 \\ \dot{x_4} &= u_2 \end{aligned} \tag{2}$$

Both the pursuer and evader agents use the same sensor model represented by a wedge (slice of a circle) centered at each agent's center of mass and have two parameters: field of view angle (rad) V_α^i and range (m) V_r^i as shown in Fig. 2b, which may differ between the agents. An agent sees its opponent if and only if the opponent is inside the agent sensor footprint. The sensor footprint of an agent i is denoted by $V^i \subseteq \mathcal{E}$ with $i \in \{p, e\}$. Therefore, the pursuer p sees the evader e if $e \in V^p$ and vice versa. The sensor footprint of an agent is also presented in Fig. 2b.

We denote the rectangular workspace by $\mathcal{E}$ as a subset of the 2D Cartesian coordinate system defined by the minimum and maximum values along each axis such that $\{X_{low}, X_{high}, Y_{low}, Y_{high}\} \subset \mathbb{R}$. $\mathcal{E}$ is assumed to be obstacle free and fully traversable. Euclidean distance between two points in $\mathcal{E}$ of arbitrary size is denoted by $d(z_1, z_2)$ with $z_1, z_2 \in \mathbb{R}^2$. All distances are defined in meters.

B. Game Formulation

Within the scope of this paper, the pursuit evasion game is defined as a POSG. However, it must be noted that without added stochastic noise, presented agent dynamics are fully deterministic as described in Eq. 1 and Eq. 2. The formal definition of a POSG is shown in 1 as defined by Terry et al. [36] inspired by Shapley [21].

Definition 1: A Partially Observable Stochastic Game (POSG) is a tuple $\langle \mathcal{S}, s_0, N, A^i, P, R^i, \Omega^i, O^i \rangle$, where:

- $\mathcal{S}$ denoting the set of possible states.
- s_0 denoting the initial state.
- N is the number of agents. The set of agents is denoted as $[N]$
- $\mathcal{A}^i$ is the set of possible actions for agent i with $i \in [N]$.
- $P : \mathcal{S} \times \prod_i \mathcal{A}^i \times \mathcal{S} \rightarrow [0, 1]$ is the transition function. It has the property that for all $s \in \mathcal{S}$, for all $(a^1, a^2, \ldots, a^N) \in \prod_i \mathcal{A}^i, \sum_{s' \in \mathcal{S}} P(s, a^1, a^2, \ldots, a^N, s') = 1$.
- $R^i : \mathcal{S} \times \prod_i \mathcal{A}^i \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function for agent i.
- Ω^i is the set of possible observations for agent i.
- $O^i : \mathcal{A}^i \times \mathcal{S} \times \Omega^i \rightarrow [0, 1]$ is the observation function. It has property that $\sum_{\omega \in \Omega^i} O^i(a, s, \omega) = 1$ for all $a \in \mathcal{A}^i$ and $s \in \mathcal{S}$

We are given a rectangular workspace $\mathcal{E}$. The positions of the agents at timestep t are defined as p_t and e_t for the pursuer and the evader, respectively. At each timestep both agents make their moves simultaneously. The pursuer captures the evader if $d(p_t, e_t) \leq 2c$ with c being each agent's radius. The pursuer wins the game if a capture occurs within an allocated timestep limit $\mathcal{T}$. Timeouts are considered evader victories.

IV. Method

The objective of each agent is to maximize its total expected reward $G^i = \sum_{t=0}^{\mathcal{T}} \gamma_t r_t^i$ over time horizon $\mathcal{T}$ where r_t^i is provided by the environment at timestep t and $\gamma \in [0, 1]$ is the discount factor. Per-agent actions are sampled from the agent policy networks $a_t^i \sim \pi_\theta^i(s_t^i)$.

A. Multi-Agent Deep Reinforcement Learning

We use the MADDPG algorithm [5] which follows the Centralized Critic, Decentralized Execution (CTDE) principle to train the agents by making them play against each other from scratch.

B. Space Representations

The hidden full state of the pursuit-evasion game between a kinematic car-like pursuer and a point-mass evader is referred as a 9-vector $s \coloneqq [x_1^p, x_2^p, x_3^p, x_4^p, x_5^p, x_1^e, x_2^e, x_3^e, x_4^e]^\mathsf{T}$. Without loss of generality, the state vector comprises of state variables presented in Section III-A. Time indices are dropped for brevity. Each agent in the pursuit-evasion game creates its own observation vector o^i through partial observations from the environment represented as 11-vectors of floating point numbers by appending a binary observation flag (represented with floating point number) indicating the visibility of the opponent and a time index. If an agent cannot observe its opponent at a timestep, corresponding values in o^i are zeroed and the binary flag is set to negative. Observation vectors are normalized to contain values in interval $[-1, 1]$. Input/action representations follow the same convention described in III-A and are represented as 2-vectors normalized to the interval $[-1, 1]$ by taking acceleration, velocity, steering angle velocity and steering angle limits. Both the state and the action spaces are continuous.

C. Reward Structure

We formulate pursuit-evasion problem as a competitive zero-sum POSG. Players have opposite rewards at each timestep, that is $r_t^p + r_t^e = 0$. At each timestep, the pursuer receives the following reward:

$$r_t^p = \begin{cases} -\lambda_{capture}, & \text{if } t \geq \mathcal{T} \\ \lambda_{capture}, & \text{if } d \leq 2c \text{ and } t < \mathcal{T} \\ -(\lambda_t + \lambda_d \cdot d), & \text{otherwise} \end{cases}$$

where $\lambda_{capture}$, λ_t, λ_d represent the coefficients for capture reward, per-timestep reward and distance reward, respectively. Each episode has two independent reset conditions, the capture condition (pursuer wins) or the timeout (evader wins). d represents the Euclidean distance between agents as described in Section III.

V. Simulation Experiments

In this section, we analyze our method by evaluating it against pursuer and evader baselines in a series of experiments. We have designed our experiments to answer the following questions: **Question 1:** How well do trained pursuer and evader agents perform against baseline methods? **Question 2:** Qualitatively, are there any interesting emerging behaviors? **Question 3:** How do the model parameters affect agents' performances?

As the pursuer baseline, we use a slightly modified pure pursuit controller [37] to handle observations where the evader is not visible. If the pursuer loses sight of the evader after observing it for at least one timestep, maximum steering velocity action is applied in arbitrary direction (positive or negative) for 25 timesteps. If the evader is not detected, the pursuer conducts a random walk, changing the input steering velocity at value every 8$^{\text{th}}$ timestep, sampled uniformly at random from the normalization interval. The velocity value is controlled to be at maximum forward velocity. For the evader baselines, we have three different baselines: a random walk algorithm for the point-mass evader, sampling linear acceleration action values from the normalization interval uniformly at random every 25 steps; a greedy evader that moves away from the pursuer upon detection and stands still otherwise; and the Rash evader from Quattrini et al. [38] which hides in an arbitrary corner until it detects the pursuer, and attempts to move to another once it detects the pursuer.

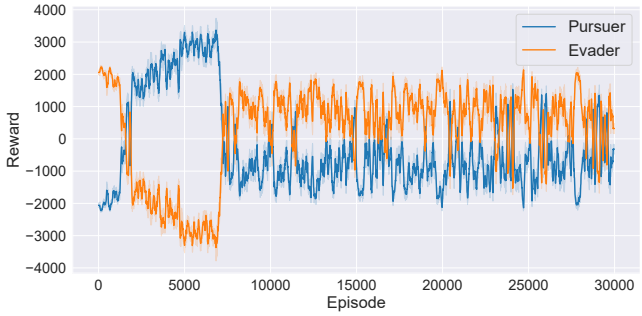


Fig. 3: **Learning regime for both agents:** Pursuer initially starts out by losing almost every episode, after a few thousand episodes the performance catches up and pursuer starts to dominate. In response, the evader develops its own counter strategies; resulting in an oscillatory reward profile with evader being slightly better on average.

A. Multi-Agent Training

During the training, both agents start from scratch and improve their performance by playing against each other over 15000 episodes. We set $\Delta t = 0.1\ s$ and maximum number of timesteps as $\mathcal{T} = 500$ with 2-frame stacking and skipping. We spawn 8 parallel vectorized environments to increase training speed with concurrency using the AgileRL Deep Reinforcement Learning Python library MADDPG and environment parallelization [39]. Actor and critic networks are both multilayer perceptrons and comprise of 2 hidden layers of size 128 each and ReLU activations. The workspace $\mathcal{E}$ is of size $(20, 20)$ in meters along the x and y-axes and is centered at $(0, 0)$. Starting positions of the agents are sampled uniformly at random within $\mathcal{E}$. Reward coefficients are set as $\lambda_{capture} = 1000$, $\lambda_d = 1$, $\lambda_t = 1$. We use a simple curriculum learning method which starts training with stronger sensor coverage for the pursuer with $V_\alpha^p = 2\pi\ rad$ and $V_r^p = 10\ m$ and linearly decays to the training model parameter over $\frac{1}{4}$ of the number of total training episodes.

For training, pursuer dynamic model parameters are set as follows: rear and front axle distances from the center-of-mass $l_f, l_r = 0.15\ m$, steering angle θ joint limits are $\pm 0.34\ rad$, steering angle velocity $\dot{\theta}$ limits are $\pm 3.2\ \frac{rad}{s}$. Minimum and maximum longitudinal velocities v are $-1\ \frac{m}{s}$ and $2.5\ \frac{m}{s}$, respectively with $\pm 2\ \frac{m}{s^2}$ maximum acceleration $\dot{v}$ value in both directions. Sensor parameters are set as $V_\alpha^p = \frac{\pi}{2}\ rad$ and $V_r^p = 7.5\ m$. Evader dynamic model parameters are: minimum and maximum velocities along the axes are $\pm 2\ \frac{m}{s}$ with $\pm 9.81\ \frac{m}{s^2}$ maximum acceleration $\dot{v}$ value in both directions.

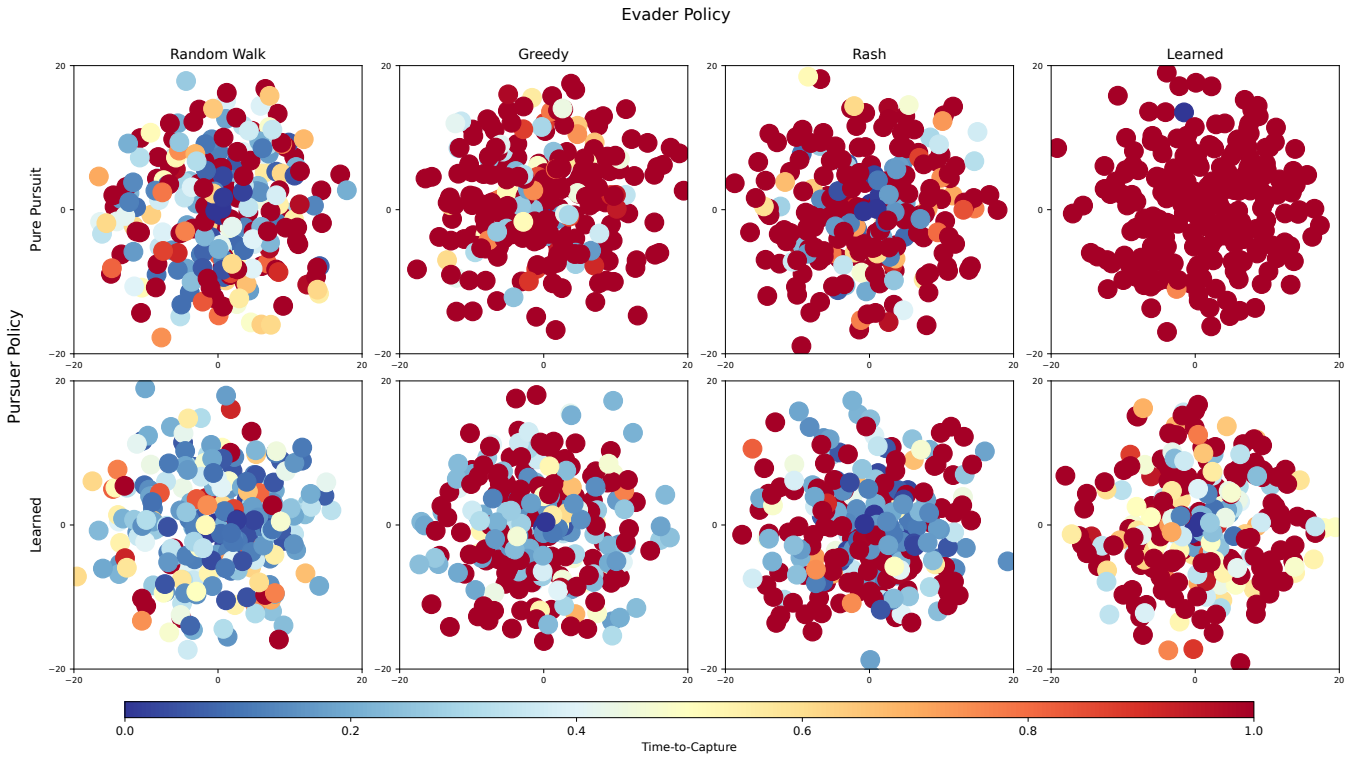


Fig. 4: **250 initial distributions sampled uniformly at random:** Initial positions are transformed with respect to the pursuer frame of reference. Colors indicate the normalized time-to-capture. Evader starting points that are farther away from pursuer have higher Time-to-Capture.

Fig. 3 shows a sample plot of averaged rewards of the agents during training. We observe a learning regime where the pursuer initially starts out losing almost every episode. Eventually the pursuer performance catches up. At that time, the evader develops its own counter strategies; resulting in an oscillatory reward profile. Maximum achievable reward by the pursuer in an individual environment before reset is $\lambda_{capture} = 1000$ (instant capture), yet the rewards are accumulated every 500 timesteps for plotting from 8 parallel environments, therefore each parallel environment can reset multiple times (indicating pursuer domination) before the evaluation timestep, resulting a higher cumulative reward. Curriculum learning method described earlier is of higher importance during the early low reward regime for the pursuer; without the curriculum, the pursuer may never be able to encounter the evader and get stuck with a very sparse reward policy, unable to learn.

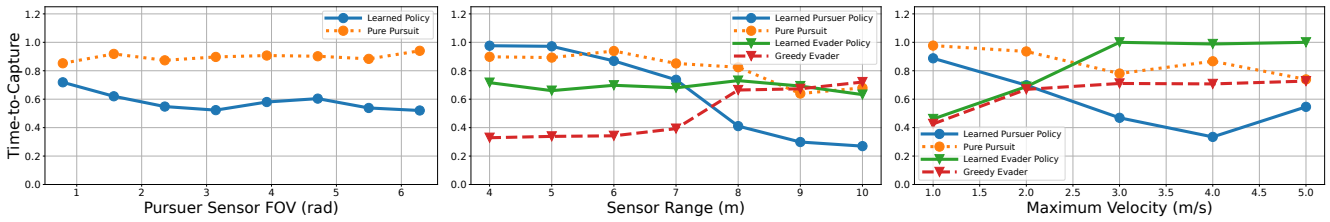
B. Performance evaluation against baselines

In our first set of experiments we investigate the performance of our learned pursuer and evader models against baseline algorithms. For multi-agent training scenario, it is not straightforward to find the best performing algorithm by evaluating algorithms based on their reward or similar metrics at the end of an episode due to the oscillatory nature of the training regime. As a more concrete example, a mediocre pursuer may achieve a deceivingly high score after training against a relatively bad evader or vice versa. To mitigate this problem, we create model checkpoints every 250 episodes and evaluate these checkpoints by setting them up against each other (one vs. 64 random samples) after training. The best performing models are selected by evaluating the average success rates and breaking ties with average capture times.

TABLE I: PURSUER CAPTURE TIMES AND RATES

	Time-to-Capture		Capture Rate	
Evader Type	Pure Pursuit	Learned	Pure Pursuit	Learned
Random Walk	0.53 (σ = 0.35)	0.35 (σ = 0.28)	0.77	0.96
Greedy	0.84 (σ = 0.28)	0.54 (σ = 0.37)	0.30	0.63
Rash	0.81 (σ = 0.33)	0.55 (σ = 0.39)	0.33	0.62
Learned	0.99 (σ = 0.09)	0.69 (σ = 0.31)	0.01	0.58

For learned pursuer and evader strategies, we tested the MADDPG algorithm against baseline pursuer and evader algorithms as well as against each other over 250 episodes. We report the results of the best performing strategies in Table I. The results are normalized between 0 and 1. For the learned pursuer strategy, a lower Time-to-Capture value and a higher capture rate are better. For the learned evader strategy, these values are the opposite. From Table I we see that the learned strategies outperform the baselines in each case and the learned evader strategy performs significantly better against the learned pursuer strategy as compared to the baseline evader strategy. Fig. 4 visualizes these results over



(a) (b) (c)

Fig. 5: **Normalized Time-to-Capture values with respect to the pursuer and evader parameters:** Pursuer policies are evaluated against greedy evader policy whereas evader policies are evaluated against learned pursuer policy. Pursuer is able to make good use of its improved sensing capabilities, evader makes rapid gains with velocity until it surpasses the default pursuer velocity.

250 initial distributions, uniformly sampled at random from $\mathcal{E}$. The top row visualizes the performance of the baseline pure pursuit where the bottom row uses the learned pursuer strategy. Initial positions are transformed with respect to the pursuer frame of reference (therefore the axes ranges are doubled). Colors indicate the normalized time-to-capture: blue indicates a quicker capture and a stronger pursuer performance; red indicates a slower capture and a stronger evader performance.

C. *Effect of model parameters*

Next, we evaluate our approach in the following scenario: pursuer policies are evaluated against greedy evader policy, whereas evader policies are evaluated against learned pursuer policy. We only evaluate by the generalized inference performance, i.e. we do not retrain the agents for the specified parameters and keep rest of the parameters in default settings as described in Section V except for the independent model parameters. Normalized Time-to-Capture values are shown in Fig. 5 with respect to the pursuer and evader parameters. In Figures 5a and 5b it is demonstrated that the learned pursuer policy is able to make good use of its improved sensing capabilities while its ability to handle better forward velocity ends up being in limited Fig. 5c. We attribute this to the relatively limited workspace area, i.e. beyond $4\frac{m}{s}$ it takes less than 5 seconds to go through an axis; higher speeds reducing agility due to braking limits and finite sensor range. The learned evader policy on the other hand does not make good use of its improved sensor range in Fig. 5b but makes rapid gains with velocity in Fig. 5c until it surpasses the default pursuer velocity. Qualitative analysis indicates that the learned evader strategy prefers hiding (close to corners) in favor of outranging pursuer sensor, only moving away if the pursuer decides to check the corner the evader is currently in. This, however comes with a significant drawback, i.e. the evader corners itself even with clear dynamic superiority and gets captured. This indicates that the qualitative nature of learned strategies are strongly correlated with model parameters.

VI. Real-World Results

For the real world experiments, a learned pursuer policy was deployed against a human controlled evader. The reinforcement learning model for the pursuer was deployed on an F1TENTH autonomous vehicle [6] with the identified system parameters from our previous work [40]. The human-controlled evader was a smaller and more nimble car-like autonomous vehicle Nvidia JetRacer [7]. Due to real world physical and system constraints, the reinforcement learning model state-space representation was slightly modified to use linear velocity and steering angle commands (as opposed to their first order derivatives), full observability was assumed (due to area being too small to properly make use of sensor footprint) and steering angles are removed from the observation state (since they are actions). A Gaussian noise with 0 mean and 5% standard deviation was added to the normalized (between -1 and 1) state and action vectors for robustness. For 6-DoF real-world state estimation, Phasespace X2E LED motion capture markers were attached to both vehicles and finite differencing was used to calculate velocities at 10 Hz frequency. ROS2 was used as the middleware for interfacing between sensors, agents and the reinforcement learning models [41]. We demonstrate trained reinforcement learning models are directly transferable to the real system and present two sample pursuit evasion sample trajectories in Fig. 6, for further details on our physical experimental results, please refer to the submitted video attachment. These experiments successfully demonstrate that the learned strategy can be executed on real systems.

VII. Conclusion

In this paper, we explored a pursuit-evasion game where the pursuer, a car with five state variables, engages with either a similar car or a point mass evader with constraints on the sensing capabilities for both agents. We formulated the game as a zero-sum Partially Observable Stochastic Game (POSG) and developed a curriculum for the MADDPG algorithm to simultaneously obtain the pursuer and evader policies. By training the players against each other, our approach sidesteps the necessity for pretrained or expert-designed evader policies. We also demonstrated that simulation-trained policies are directly transferable to real autonomous agents.

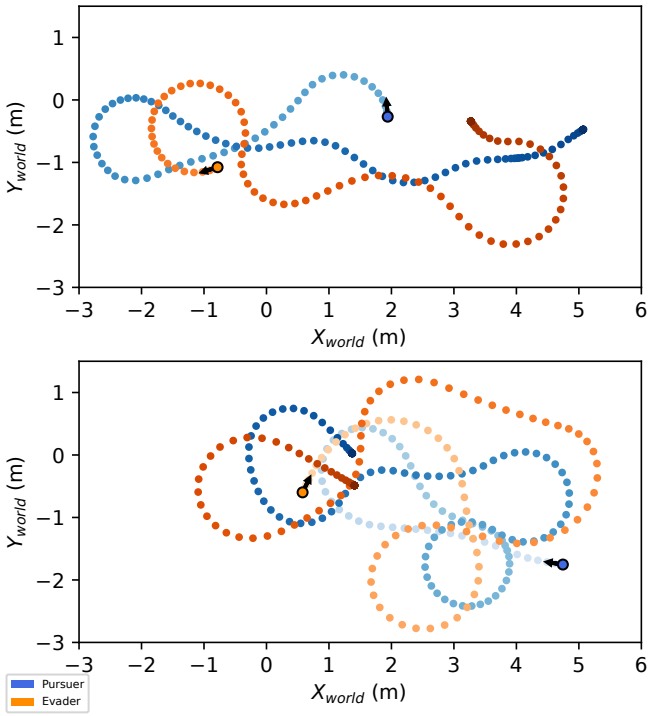


Fig. 6: **Real system trajectories:** We demonstrate trained reinforcement learning models are directly transferable. Lighter colors have earlier timestamps, darker colors are later. For further details on our physical experimental results, please refer to the submitted video attachment.

Through this exploration, we show the potential of multi-agent reinforcement learning in navigating complex pursuit-evasion scenarios within dynamic environments.

There are multiple avenues for future research: in our current formulation, we did not model uncertainty in the players own states. Preliminary experiments indicate that the learned strategies are robust to small errors in state. As the arena gets bigger and if the players do not have perfect global sensors, the uncertainty in state estimation may need to be addressed explicitly. Furthermore, if the game takes place in complex environments, the search component of the game becomes prominent. It is unclear if current RL methods will be able to handle such scenarios in which there will be long sequences with no reward. We will explore these exciting, yet challenging, issues in our future work.

References

[1] L. J. Guibas, J.-C. Latombe, S. M. LaValle, D. Lin, and R. Motwani, "A visibility-based pursuit-evasion problem," *International Journal of Computational Geometry & Applications*, vol. 9, no. 04n05, pp. 471–493, 1999.

[2] V. Isler, S. Kannan, and S. Khanna, "Randomized pursuit-evasion in a polygonal environment," *IEEE Transactions on Robotics*, vol. 21, no. 5, pp. 875–884, 2005.

[3] R. Isaacs, *Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization*. Courier Corporation, 1999, originally published in 1965.

[4] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[5] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.

[6] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1tenth: An open-source evaluation environment for continuous control and reinforcement learning," *Proceedings of Machine Learning Research*, vol. 123, 2020.

[7] NVIDIA-AI-IOT, "jetracer." [Online]. Available: https://github.com/NVIDIA-AI-IOT/jetracer

[8] T. H. Chung, G. A. Hollinger, and V. Isler, "Search and pursuit-evasion in mobile robotics: A survey," *Autonomous robots*, vol. 31, pp. 299–316, 2011.

[9] I. Exarchos, P. Tsiotras, and M. Pachter, "On the suicidal pedestrian differential game," *Dynamic Games and Applications*, vol. 5, pp. 297–317, 2015.

[10] U. Ruiz and R. Murrieta-Cid, "A differential pursuit/evasion game of capture between an omnidirectional agent and a differential drive robot, and their winning roles," *International Journal of Control*, vol. 89, no. 11, pp. 2169–2184, 2016.

[11] W. L. Scott and N. E. Leonard, "Optimal evasive strategies for multiple interacting agents with motion constraints," *Automatica*, vol. 94, pp. 26–34, 2018.

[12] C. De Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulić, "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4552–4559, 2021.

[13] H. Yang, P. Ge, J. Cao, Y. Yang, and Y. Liu, "Large scale pursuit-evasion under collision avoidance using deep reinforcement learning," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2232–2239.

[14] I. Suzuki and M. Yamashita, "Searching for a mobile intruder in a polygonal region," *SIAM Journal on computing*, vol. 21, no. 5, pp. 863–888, 1992.

[15] B. P. Gerkey, S. Thrun, and G. Gordon, "Visibility-based pursuit-evasion with limited field of view," *The International Journal of Robotics Research*, vol. 25, no. 4, pp. 299–315, 2006.

[16] N. M. Stiffler, A. Kolling, and J. M. O'Kane, "Persistent pursuit-evasion: The case of the preoccupied pursuer," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 5027–5034.

[17] D. Shishika and V. Kumar, "Local-game decomposition for multiplayer perimeter-defense problem," in *2018 IEEE conference on decision and control (CDC)*. IEEE, 2018, pp. 2093–2100.

[18] N. Noori and V. Isler, "The lion and man game on polyhedral surfaces with boundary," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014, pp. 1769–1774.

[19] S. Engin, Q. Jiang, and V. Isler, "Learning to play pursuit-evasion with visibility constraints," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 3858–3863.

[20] A. Bajcsy, A. Loquercio, A. Kumar, and J. Malik, "Learning vision-based pursuit-evasion robot policies," *arXiv preprint arXiv:2308.16185*, 2023.

[21] L. S. Shapley, "Stochastic games," *Proceedings of the national academy of sciences*, vol. 39, no. 10, pp. 1095–1100, 1953.

[22] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine learning proceedings 1994*. Elsevier, 1994, pp. 157–163.

[23] I. M. Mitchell, A. M. Bayen, and C. J. Tomlin, "A time-dependent hamilton-jacobi formulation of reachable sets for continuous dynamic games," *IEEE Transactions on automatic control*, vol. 50, no. 7, pp. 947–957, 2005.

[24] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," *Handbook of reinforcement learning and control*, pp. 321–384, 2021.

[25] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2017, pp. 2817–2826.

[26] L. Buşoniu, R. Babuška, and B. De Schutter, "Multi-agent reinforcement learning: An overview," *Innovations in multi-agent systems and applications-1*, pp. 183–221, 2010.

[27] J. Ackermann, V. Gabler, T. Osa, and M. Sugiyama, "Reducing overestimation bias in multi-agent domains using double centralized critics," *arXiv preprint arXiv:1910.01465*, 2019.

[28] C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. Torr, M. Sun, and S. Whiteson, "Is independent learning all you need in the starcraft multi-agent challenge?" *arXiv preprint arXiv:2011.09533*, 2020.

[29] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 7234–7284, 2020.

[30] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, *et al.*, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[31] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, *et al.*, "Dota 2 with large scale deep reinforcement learning," *arXiv preprint arXiv:1912.06680*, 2019.

[32] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[33] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[34] A. Brunnbauer, L. Berducci, A. Brandstátter, M. Lechner, R. Hasani, D. Rus, and R. Grosu, "Latent imagination facilitates zero-shot transfer in autonomous racing," in *2022 international conference on robotics and automation (ICRA)*. IEEE, 2022, pp. 7513–7520.

[35] M. Althoff, M. Koschi, and S. Manzinger, "Commonroad: Composable benchmarks for motion planning on roads," in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 719–726.

[36] J. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, C. Dieffendahl, C. Horsch, R. Perez-Vicente, *et al.*, "Pettingzoo: Gym for multi-agent reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 15 032–15 043, 2021.

[37] R. C. Coulter *et al.*, *Implementation of the pure pursuit path tracking algorithm*. Carnegie Mellon University, The Robotics Institute, 1992.

[38] A. Quattrini Li, R. Fioratto, F. Amigoni, and V. Isler, "A search-based approach to solve pursuit-evasion games with limited visibility in polygonal environments," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, 2018, pp. 1693–1701.

[39] N. Ustaran-Anderegg and M. Pratt, "AgileRL." [Online]. Available: https://github.com/AgileRL/AgileRL

[40] B. M. Gonultas, P. Mukherjee, O. G. Poyrazoglu, and V. Isler, "System identification and control of front-steered ackermann vehicles through differentiable physics," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 4347–4353.

[41] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074